

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#04)

Use of Venus Simulator for Register Level Execution of RISC-V Control Instructions

Student Name: Muhammad Abdullah Tanveer Roll No: 24i-6508 Section: A

Date performed: 15.09.2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1 :

Description:
This task was to decide which of the control instructions are real hardware and which pseudoinstructions. Pseudo-instructions are simple instructions that are given to the programmer, but, during compilation, they are expanded to real instructions. We simulated these in the Venus simulator, and correctly interpreted their classifications.
Code:
Actual Instructions: beq, blt, bge, bne, jal, and jalr. Pseudo-Instructions: ble (expands to bge t1, t0, target), bgt (expands to blt t1, t0, target), and j (expands to jal x0, target).
Output:
beq = Actual Instruction blt = Actual Instruction ble = Pseudo-Instruction bgt = Pseudo-Instruction bge = Actual Instruction bne = Actual Instruction j = Pseudo-Instruction jal = Actual Instruction jalr = Actual Instruction

Task 2:

Description:

In this task, we compared how conditional branching and unconditional jumping work using beq and j. The beq instruction only jumps if a specific condition is met, while j always jumps no matter what. We executed and simulated this in the Venus simulator and correctly interpreted the execution flow.

Code:

```
addi s0, x0, 5
addi s1, x0, 5
beq s0, s1, target
addi s2, x0, 10
target:
j end
addi s2, x0, 20
end: |
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00500413	addi x8 x0 5	addi s0, x0, 5
0x4	0x00500493	addi x9 x0 5	addi s1, x0, 5
0x8	0x00940463	beq x8 x9 8	beq s0, s1, target
0xc	0x00A00913	addi x18 x0 10	addi s2, x0, 10
0x10	0x0080006F	jal x0 8	j end
0x14	0x01400913	addi x18 x0 20	addi s2, x0, 20

Task 3:

Description:

In this task, we ran two short programs to see how branch decisions change which lines of code get executed. The first program ran the addition path because the numbers were different, while the second jumped to the subtraction path because the numbers matched. We executed and simulated both in the Venus simulator and correctly interpreted the final register values.

Code:

Code 1

```
addi s0, s0, 4 #Add 4 to s0 (s0 = 4)
addi s1, s1, 6 #Add 6 to s1 (s1 = 6)
beq s0, s1, Fin # Check if s0 == s1 (False: 4 != 6, moves to next line)
True:
    add s2, s1, s0 # Add s0 and s1 (s2 = 10)
j end # Jump to end label
Fin:
sub s2, s1, s0 # Subtract s0 from s1 (Skipped)
end:
```

Code 2

```
addi s0, x0, 4 # Add 4 to s0 (s0 = 4)
addi s1, x0, 4 #Add 4 to s1 (s1 = 4)
beq s0, s1, Fin #Check if s0 == s1 (True: 4 == 4, branches to Fin)
True:
    add s2, s1, s0 # Add s0 and s1 (Skipped)
j end #Jump to end (Skipped)
Fin:
sub s2, s1, s0 #Subtract s0 from s1 (s2 = 4 - 4 = 0)
end:
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00440413	addi x8 x8 4	addi s0, s0, 4 #Add 4 to s0 (s0 = 4)
0x4	0x00648493	addi x9 x9 6	addi s1, s1, 6 #Add 6 to s1 (s1 = 6)
0x8	0x00940663	beq x8 x9 12	beq s0, s1, Fin # Check if s0 == s1 (False: 4 != 6, moves to next line)
0xc	0x00848933	add x18 x9 x8	add s2, s1, s0 # Add s0 and s1 (s2 = 10)
0x10	0x0080006F	jal x0 8	j end # Jump to end label
0x14	0x40848933	sub x18 x9 x8	sub s2, s1, s0 # Subtract s0 from s1 (Skipped)
PC	Machine Code	Basic Code	Original Code
0x0	0x00400413	addi x8 x0 4	addi s0, x0, 4 # Add 4 to s0 (s0 = 4)
0x4	0x00400493	addi x9 x0 4	addi s1, x0, 4 #Add 4 to s1 (s1 = 4)
0x8	0x00940663	beq x8 x9 12	beq s0, s1, Fin #Check if s0 == s1 (True: 4 == 4, branches to Fin)
0xc	0x00848933	add x18 x9 x8	add s2, s1, s0 # Add s0 and s1 (Skipped)
0x10	0x0080006F	jal x0 8	j end #Jump to end (Skipped)
0x14	0x40848933	sub x18 x9 x8	sub s2, s1, s0 #Subtract s0 from s1 (s2 = 4 - 4 = 0)

Task 4:

Description:

For this task, we used the beq instruction to compare two registers and started using conditional branching. The values were not equal so the program did not execute the subtraction block, but did execute the addition block, and went to the end. We implemented and ran this code in the Venus simulator, and we obtained the correct results in the control path and register values.

Code:

```
addi s0,x0,5
addi s1,x0,10
addi s3,x0,4
addi s4,x0,3
beq s0,s1,TRUE # beq means branch if equal
add s2,s3,s4
j END
TRUE:
sub s2,s3,s4
END:
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00500413	addi x8 x0 5	addi s0,x0,5
0x4	0x00A00493	addi x9 x0 10	addi s1,x0,10
0x8	0x00400993	addi x19 x0 4	addi s3,x0,4
0xc	0x00300A13	addi x20 x0 3	addi s4,x0,3
0x10	0x00940663	beq x8 x9 12	beq s0,s1,TRUE # beq means branch if equal
0x14	0x01498933	add x18 x19 x20	add s2,s3,s4
0x18	0x0080006F	jal x0 8	j END
0x1c	0x41498933	sub x18 x19 x20	sub s2,s3,s4

Task 5:

Description:			
In this task, bitwise logical left shift operations have been performed on a register, using the slli instruction. We saw that repeatedly moving left by one position doubles the decimal value of the register by 10. We ran and tested this code using the Venus simulator and the outputs of the registers and bit shifts were explained.			
Code:			
<pre>addi s0, x0, 2 slli s0, s0, 1 slli s0, s0, 1</pre>			
Output:			
PC	Machine Code	Basic Code	Original Code
0x0	0x00200413	addi x8 x0 2	addi s0, x0, 2
0x4	0x00141413	slli x8 x8 1	slli s0, s0, 1
0x8	0x00141413	slli x8 x8 1	slli s0, s0, 1

Task 6:

Description:			
In this exercise, we constructed an elementary counter loop with blt instruction that is designed to execute repeatedly until a desired number was reached. The registers were incremented each repetition until it failed the condition. This code was executed and simulated in the Venus simulator, and the iterations of the loop and changes in the registers were correctly interpreted.			
Code:			

```
addi s0,x0,0
addi s1,x0,1
Loop:
addi s0,s0,1
addi s1,s1,1
addi t0,x0,6
blt s1,t0,Loop
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00000413	addi x8 x0 0	addi s0,x0,0
0x4	0x00100493	addi x9 x0 1	addi s1,x0,1
0x8	0x00140413	addi x8 x8 1	addi s0,s0,1
0xc	0x00148493	addi x9 x9 1	addi s1,s1,1
0x10	0x00600293	addi x5 x0 6	addi t0,x0,6
0x14	0xFE54CAE3	blt x9 x5 -12	blt s1,t0,Loop

Task 7:

Description:

In this task, we calculated the address of array memory by computing the offset in bytes for the array and stored the calculated values in data memory using the sw instruction. A piece of code that performs a loop five times to write the values in turn into neighbouring memory locations. We performed and simulated this code in the simulator venus and the memory updates and address calculations were interpreted correctly.

Code:

```
.data
num: .word 0,0,0,0,0,0,0,0,0,0

.text
main:
    addi s0,x0,0
    la s1,num

Loop:
    addi t0,s0,5
    slli t1,s0,2
    add t2,s1,t1
    sw t0,0(t2)
    addi s0,s0,1
    addi t3,x0,5
    blt s0,t3,Loop
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00000413	addi x8 x0 0	addi s0,x0,0
0x4	0x10000497	auipc x9 65536	la s1,num
0x8	0xFFC48493	addi x9 x9 -4	la s1,num
0xc	0x00540293	addi x5 x8 5	addi t0,s0,5
0x10	0x00241313	slli x6 x8 2	slli t1,s0,2
0x14	0x006483B3	add x7 x9 x6	add t2,s1,t1
0x18	0x0053A023	sw x5 0(x7)	sw t0,0(t2)
0x1c	0x00140413	addi x8 x8 1	addi s0,s0,1
0x20	0x00500E13	addi x28 x0 5	addi t3,x0,5
0x24	0xFFC444E3	blt x8 x28 -24	blt s0,t3,Loop

Task 8:

Description:

For this task, we designed a nested loop structure, in which the inner loop was executed several times for every iteration of the outer loop. The inner addition was performed 6 times, before both of the loop conditions were met. We implemented and ran this code in the Venus simulator and understood it step-by-step. Update the registers for each iteration.

Code:

```

addi s0,x0,0
addi s1,x0,0

Outer:
addi s2,x0,0

Inner:
add s0,s1,s2
addi s2,s2,1

addi t0,x0,2
blt s2,t0,Inner
addi s1,s1,1
addi t1,x0,3
blt s1,t1,Outer

```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00000413	addi x8 x0 0	addi s0,x0,0
0x4	0x00000493	addi x9 x0 0	addi s1,x0,0
0x8	0x00000913	addi x18 x0 0	addi s2,x0,0
0xc	0x01248433	add x8 x9 x18	add s0,s1,s2
0x10	0x00190913	addi x18 x18 1	addi s2,s2,1
0x14	0x00200293	addi x5 x0 2	addi t0,x0,2
0x18	0xFE594AE3	blt x18 x5 -12	blt s2,t0,Inner
0x1c	0x00148493	addi x9 x9 1	addi s1,s1,1
0x20	0x00300313	addi x6 x0 3	addi t1,x0,3
0x24	0xFE64C2E3	blt x9 x6 -28	blt s1,t1,Outer

--